Customer Churn Prediction System

Complete Technical Deep-Dive Report

End-to-End Machine Learning System for Telecom Customer Retention

Generated: July 12, 2026 at 05:58 PM

Best Model: Random Forest | ROC-AUC: 0.8362

Dataset: IBM Telco Customer Churn | 7,021 Records | 24 Features

Stack: Python 3.12 | Scikit-Learn | XGBoost | SMOTE | SQLite | Streamlit

Table of Contents

1. Business Problem & Motivation
2. Dataset Deep-Dive — All 20 Features
3. Project Architecture & File Structure
4. Data Preprocessing Pipeline — 7 Stages
5. Feature Engineering — 5 Custom Features
6. SMOTE — Handling Class Imbalance
7. Exploratory Data Analysis — 9 Charts
8. SQL Analytics Layer
9. Machine Learning Models — How They Work
   - 9.1. Logistic Regression
   - 9.2. Random Forest (Winner)
   - 9.3. XGBoost Gradient Boosting
10. Model Evaluation Metrics Explained
11. Live Model Performance Results
12. Feature Importance & Business Insights
13. Prediction Engine & Generic Pipeline
14. Streamlit Dashboard — All 5 Pages
15. Key Business Findings & Recommendations
16. Files & Artifacts Reference
17. Technology Stack & Future Improvements

1. Business Problem & Motivation

Why predicting customer churn is worth millions of dollars

In the telecom industry, **customer churn** — when a subscriber cancels or switches provider — is one of the costliest business problems. Research shows acquiring a new customer costs **5–7x more** than retaining one. A single percentage point reduction in churn rate can save hundreds of thousands of dollars annually for a mid-sized provider.

This system uses Machine Learning to **predict which customers will churn before they do** — shifting from reactive damage control to proactive retention. The model scores every customer with a churn probability (0–100%), explains the key risk factors, and generates personalised retention recommendations.

System Capabilities

- **Predict** — Churn probability score (0–100%) for any customer
- **Explain** — Top risk factors driving each prediction
- **Quantify** — Revenue at risk via SQL analytics ($138,529/month in this dataset)
- **Recommend** — Personalised retention interventions
- **Generalise** — Works on any CSV dataset via the Generic Pipeline
- **Visualise** — Full interactive dashboard with 5 pages

Live Business Metrics from This Run

Metric	Value	Metric	Value
Total Customers	7,021	Churned Customers	1,857
Overall Churn Rate	26.45%	Revenue at Risk/Month	$138,529
Month-to-Month Churn	42.64%	2-Year Contract Churn	2.83%
Electronic Check Churn	45.15%	Fiber Optic Churn	41%+
Best Model ROC-AUC	0.8362	5-Fold CV AUC	0.9012

2. Dataset Deep-Dive

IBM Telco Customer Churn Dataset — 7,043 customers, 20 features

The **IBM Telco Customer Churn Dataset** is a widely-used benchmark in the data science community. It captures a realistic cross-section of telecom customer behaviour, covering demographics, service subscriptions, billing, payment behaviour, and whether each customer churned. After cleaning, we work with **7,021 rows and 20 features** (19 predictors + 1 target).

Feature	Type	Values / Range	Business Meaning
gender	Binary	Male / Female	Customer gender
SeniorCitizen	Binary	0 / 1	Senior citizen flag (16% of customers)
Partner	Binary	Yes / No	Has a domestic partner
Dependents	Binary	Yes / No	Has children or dependents
tenure	Numeric	0 – 72 months	How long they have been a customer
PhoneService	Binary	Yes / No	Has a phone line
MultipleLines	Categorical	Yes/No/No phone svc	Has multiple phone lines
InternetService	Categorical	DSL/Fiber optic/No	Type of internet service (key churn driver)
OnlineSecurity	Binary	Yes/No/No internet svc	Security add-on subscription
OnlineBackup	Binary	Yes/No/No internet svc	Backup add-on subscription
DeviceProtection	Binary	Yes/No/No internet svc	Device protection add-on
TechSupport	Binary	Yes/No/No internet svc	Tech support subscription
StreamingTV	Binary	Yes/No/No internet svc	Streaming TV subscription
StreamingMovies	Binary	Yes/No/No internet svc	Streaming movies subscription
Contract	Ordinal	Month-to-month/1yr/2yr	Contract commitment level (biggest churn driver)
PaperlessBilling	Binary	Yes / No	Digital billing preference
PaymentMethod	Categorical	4 payment types	How the customer pays (electronic check = highest risk)
MonthlyCharges	Numeric	$18.25 – $118.75	Monthly billing amount
TotalCharges	Numeric	$0 – $8,684.80	Cumulative lifetime billing amount
Churn	**TARGET**	**Yes=1 / No=0**	**Did the customer cancel? (what we predict)**

3. Project Architecture & File Structure

How all components connect

The system follows a clean, modular, pipeline-oriented architecture. Each Python module has a single responsibility and can be run independently. The master orchestrator **main.py** calls them in sequence.

```
Customer_Churn_Prediction_System/
  main.py                     <- Master pipeline (run this first)
  requirements.txt            <- All Python dependencies
  generate_pdf_report.py      <- This PDF report generator

  src/
    utils.py                  <- Shared: paths, logging, plot styles
    preprocessing.py          <- Stage 1-7: load,clean,encode,scale,split
    feature_engineering.py    <- 5 engineered features + SMOTE
    eda.py                    <- 9 EDA visualisation functions
    sql_integration.py        <- SQLite DB creation + 7 analytics queries
    train_model.py            <- Train LR, RF, XGBoost; save best model
    predict.py                <- Single + batch customer inference
    generic_pipeline.py       <- Schema-agnostic universal ML pipeline

  dashboard/
    app.py                    <- Streamlit 5-page web dashboard (~58KB)

  data/
    raw/telco_churn_raw.csv   <- Original IBM dataset (7043 rows)
    processed/                <- train.csv, test.csv, churn_processed.csv
    churn.db                  <- SQLite analytics database

  models/
    best_model.pkl            <- Saved best model (joblib serialized)
    random_forest.pkl         <- Random Forest model
    logistic_regression.pkl   <- Logistic Regression model
    xgboost.pkl               <- XGBoost model
    model_results.json        <- All model metrics
    feature_importance.csv    <- Feature importance ranking

  reports/figures/            <- 13 generated chart PNGs
```

Step	Module	Input	Output
1	preprocessing.py	Raw CSV (7,043 rows)	Encoded X_train, X_test, y_train, y_test
2	feature_engineering.py	Encoded DataFrame	24-feature set + SMOTE balanced (8,262 rows)
3	eda.py	Raw + encoded DataFrame	9 PNG chart files in reports/figures/
4	sql_integration.py	Processed DataFrame	SQLite DB + 7 analytics query results
5	train_model.py	X_train_bal, y_train_bal	3 trained models + best_model.pkl + charts
6	predict.py	Customer features dict	Churn probability + risk level + recommendations

7	dashboard/app.py	All of the above	Interactive 5-page web UI at localhost:8501

4. Data Preprocessing Pipeline

7 sequential stages transforming raw CSV into ML-ready feature matrices

Implemented in **src/preprocessing.py**. Every transformation follows the golden rule: *fit only on training data, transform both train and test*. This prevents data leakage — a common mistake that leads to optimistic but unrealistic model performance estimates.

Stage 1 — Data Loading (load_data)

- Checks if telco_churn_raw.csv exists locally (cache-first approach for speed)
- If not found: downloads from IBM's public GitHub repository via urllib
- Fallback: generates a statistically realistic synthetic dataset if offline
- Raw data loaded: 7,043 rows x 21 columns

Stage 2 — Data Cleaning (clean_data)

- Drops customerID — a unique identifier with no predictive value
- Removes 22 exact duplicate rows (identical on all 21 columns)
- Fixes TotalCharges column: blank string entries (11 new customers with tenure=0) converted to NaN, then imputed as MonthlyCharges x tenure
- Encodes target: 'Yes' -> 1, 'No' -> 0 for the Churn column
- Ensures numeric dtype for: tenure, MonthlyCharges, TotalCharges, SeniorCitizen
- Final shape after cleaning: 7,021 rows x 20 columns

Stage 3 — Outlier Handling (handle_outliers)

- Method: Winsorisation (capping, not removal) — preserves all rows
- MonthlyCharges: capped at 1st percentile ($19.20) and 99th percentile ($114.74)
- TotalCharges: capped at $19.75 and $8,040.51
- Why capping over removal? No data is lost. Extreme values are brought in range without distorting the dataset size.
- Why not Z-score? IQR-based capping is more robust to non-normal distributions

Stage 4 — Feature Encoding (encode_features)

- Binary Yes/No columns (Partner, Dependents, PhoneService, PaperlessBilling): mapped to 0/1 using dict lookup
- Service columns (OnlineSecurity, OnlineBackup, etc.): 'No internet service' and 'No phone service' treated as No=0, else 1
- Contract (ordinal): Month-to-month=0, One year=1, Two year=2 — order preserved intentionally

- gender, InternetService, PaymentMethod: LabelEncoder (alphabetical integer assignment)

Stage 5 — Train/Test Split (split_data)

- 80/20 split: 5,616 training rows and 1,405 test rows
- Stratified by Churn — ensures same ~26% churn rate in both sets
- random_state=42 — fully reproducible across runs
- Stratification is critical: without it, random splits could create unrepresentative subsets

Stage 6 — Feature Scaling (scale_features)

- StandardScaler applied to: tenure, MonthlyCharges, TotalCharges
- Formula: z = (x - mean) / std — transforms each column to mean=0, std=1
- Scaler FITTED on X_train only, then APPLIED (transform only) to X_test
- Prevents leakage: test set statistics should never influence training normalization
- Why scale? Logistic Regression's L-BFGS solver converges poorly on unscaled features. Also helps regularization work correctly.

Stage 7 — Artifact Persistence

- churn_processed.csv: full cleaned+encoded dataset (347 KB)
- train.csv: 5,616 training samples with Churn label
- test.csv: 1,405 test samples with Churn label
- All downstream modules can reload these without re-running preprocessing

5. Feature Engineering

5 domain-specific features created from raw columns — the #1 and #2 most predictive features

Feature engineering is where domain expertise translates into model performance. The 5 features created here ranked **#1 and #2 in feature importance** — outranking all 19 original raw columns. This demonstrates that thoughtful feature creation is often more valuable than algorithmic complexity.

Feature Name	Formula	Output Range	Why It Works
ChargesPerTenure	MonthlyCharges / (tenure + 1)	0.28 – 114.74	High charges early in tenure signals value-for-money dissatisfact
ServiceCount	Sum of 6 add-on service columns (0 or 1 each)	0 – 6	Each additional service creates switching cost. A customer with c
IsHighValue	1 if MonthlyCharges >= 75th percentile, else 0	0 or 1	Top revenue customers may have higher service expectations. If
HasLoyalty	1 if tenure > 24 months, else 0	0 or 1	Customers who have stayed over 2 years exhibit a strong retenti
RiskScore	0.4 * contract_risk + 0.3 * charges_norm + 0.3 * (1 - tenure_norm)	0.0 – 1.0	Composite weighted signal combining the 3 strongest churn pred

RiskScore Decomposed

RiskScore is the most analytically sophisticated feature. Each component is normalized to [0,1] before weighting:

- contract_risk: Month-to-month=1.0, One year=0.5, Two year=0.0 (linear mapping)
- charges_norm: (MonthlyCharges - min) / (max - min) — 0=cheapest plan, 1=most expensive
- tenure_norm: 1 - (tenure - min) / (max - min) — 0=long customer, 1=brand new customer
- Weights: Contract (40%) is weighted highest because it is the strongest predictor. Charges and tenure tied at 30% each.

6. SMOTE — Handling Class Imbalance

Synthetic Minority Over-sampling Technique

The dataset has a **3:1 class imbalance**: 74% No Churn vs 26% Churn. Without intervention, models will over-optimize for the majority class.

The naive baseline problem: A model that predicts 'No Churn' for every customer achieves 74% accuracy — but has 0% recall on actual churners. This is useless for retention purposes. We need the model to correctly identify churners (high recall), even at the cost of some false alarms.

How SMOTE Works — Step by Step

Step 1 — Sample Selection: For each minority-class (Churn=1) training sample, identify its K nearest neighbors in feature space (K=5 by default, using Euclidean distance).

Step 2 — Neighbor Selection: Randomly select one of those K neighbors.

Step 3 — Synthetic Point Creation: Generate a new synthetic sample at a random position along the line segment between the original sample and the chosen neighbor: synthetic = original + lambda * (neighbor - original), where lambda ~ Uniform(0,1).

Step 4 — Repeat: Repeat Steps 1-3 until the minority class reaches the target balance (50/50 in this implementation).

Result: Training set: 5,616 rows -> 8,262 rows. Churn rate: 26.44% -> 50.00%. Each new sample is a plausible interpolation between real churners.

Critical rule: SMOTE is applied *only to the training set*. The test set retains its original 26% churn rate. If SMOTE were applied before splitting, synthetic test samples would make evaluation unrealistically optimistic.

Why Not Class Weights Instead?

- Class weights (e.g., class_weight='balanced') tell the model to penalize minority-class errors more — it adjusts the loss function
- SMOTE physically creates new training examples — it changes the data distribution itself
- SMOTE often produces better recall: the model sees more diverse minority-class examples in feature space
- Both are used in this project: SMOTE for preprocessing + scale_pos_weight=3 in XGBoost as a backup signal

7. Exploratory Data Analysis

9 publication-quality visualisations to understand churn patterns

All 9 EDA figures are generated by **src/eda.py** using Matplotlib and Seaborn on a dark theme (#0F172A background). They are saved to **reports/figures/** at 150 DPI and embedded in the dashboard.

1. Churn Distribution

Pie chart showing 26.45% churn vs 73.55% retention. Count bar confirms 1,857 churned and 5,164 retained. Sets the baseline for all downstream analysis.

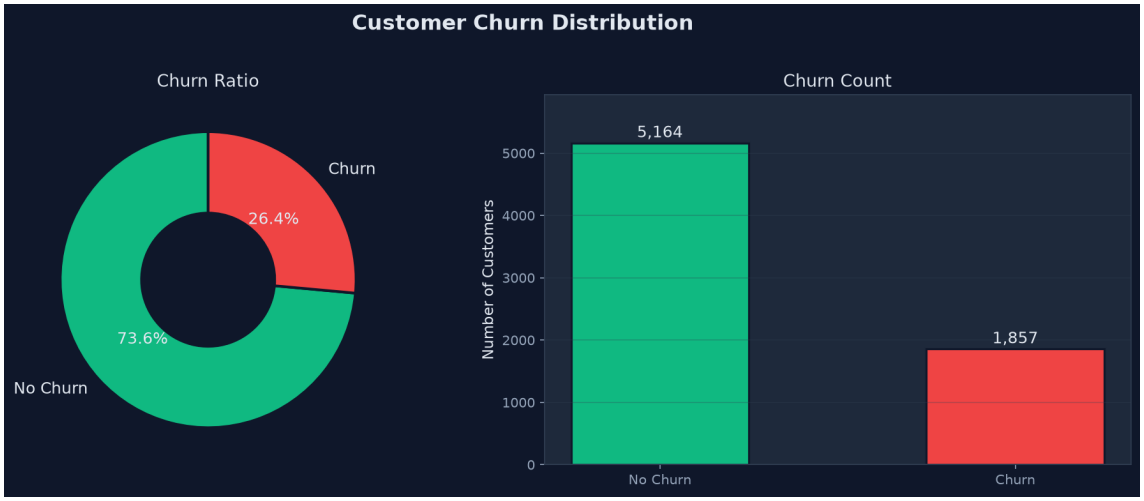


Figure saved at: reports/figures/01_churn_distribution.png

2. Customer Tenure Analysis

Left: histogram showing churned customers peak at 0-10 months while retained customers spread evenly. Right: boxplot confirms median tenure for churners is ~10 months vs ~38 months for retained. The tenure gap is the single clearest visual signal.

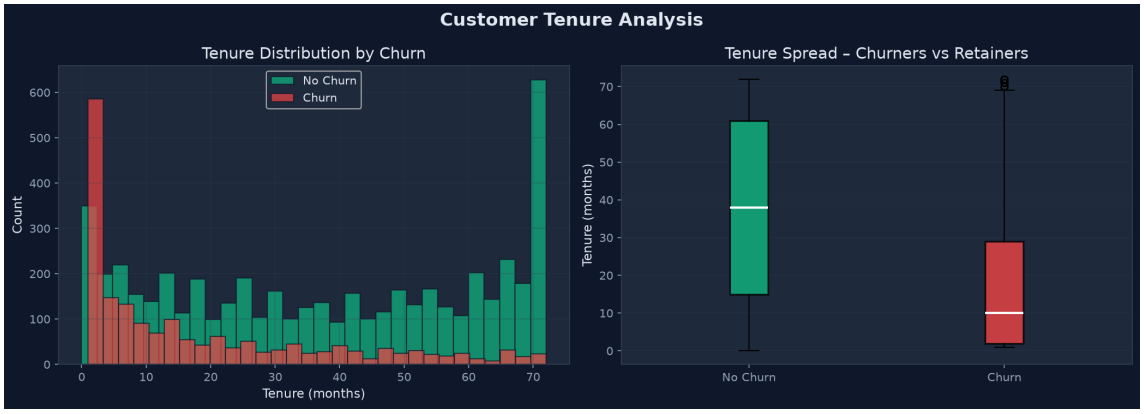


Figure saved at: reports/figures/02_tenure_analysis.png

3. Contract Type Analysis

Stacked bar by contract type. Month-to-month accounts for 55% of customers but 88% of churners. Two-year contracts barely register on the churn scale (2.83%).

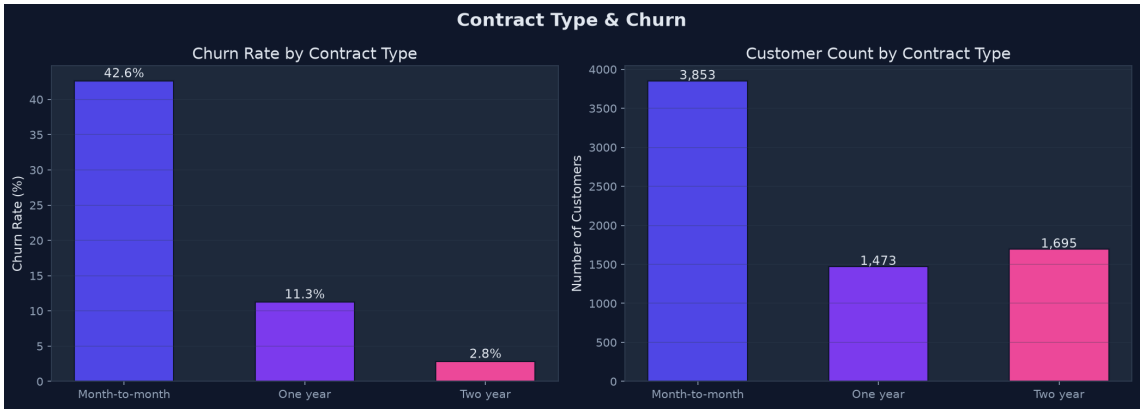


Figure saved at: reports/figures/03_contract_analysis.png

4. Monthly Charges Distribution

KDE (kernel density) plots overlaid for churned vs retained. Churners cluster heavily in the $65-$100 range. Retained customers are more evenly spread. Price sensitivity is clearly higher among churners.

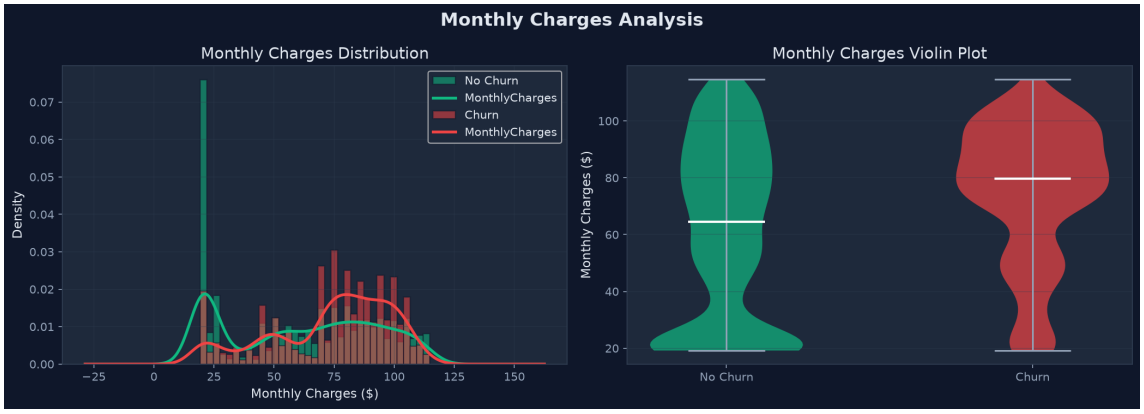


Figure saved at: reports/figures/04_monthly_charges.png

5. Feature Correlation Heatmap

Full correlation matrix of all encoded features. Key findings: Churn most negatively correlated with tenure (-0.35) and Contract (-0.24). Churn positively correlated with MonthlyCharges (+0.19). TotalCharges and tenure are highly correlated (0.83) — expected since TotalCharges = MonthlyCharges x tenure.

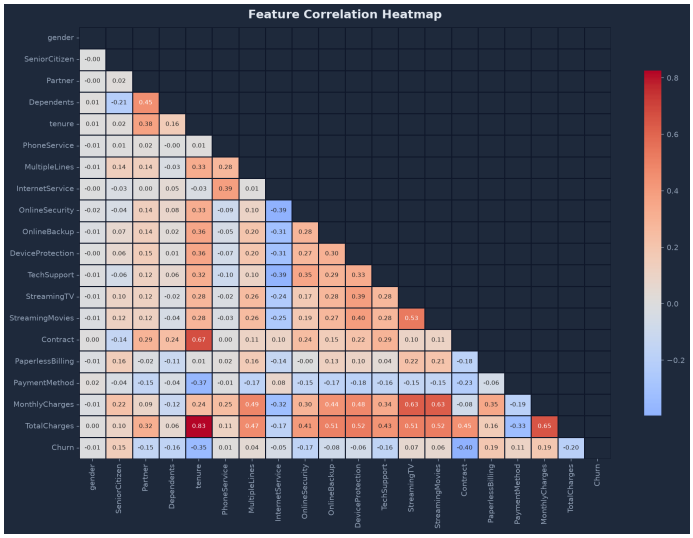


Figure saved at: reports/figures/05_correlation_heatmap.png

6. Churn by Demographics

Senior citizens have a churn rate of ~42% vs ~23% for non-seniors — nearly 2x higher. Gender shows almost no difference. Having dependents significantly reduces churn (loyal family subscribers).

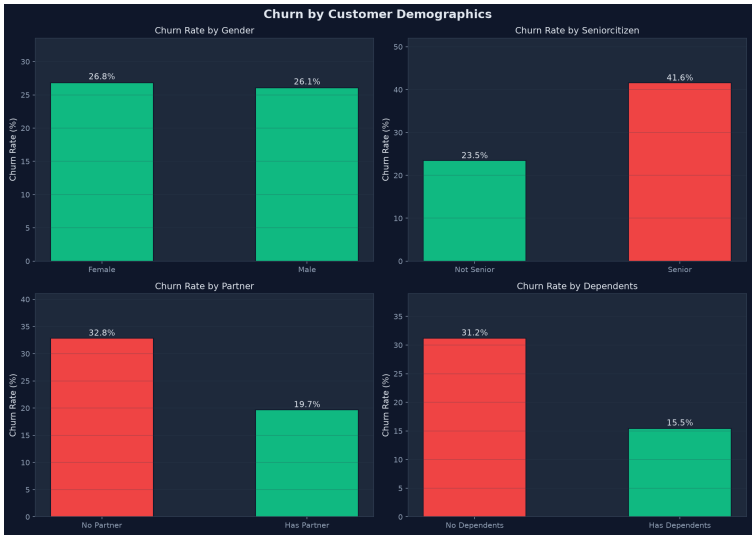


Figure saved at: reports/figures/06_churn_demographics.png

7. Service & Payment Analysis

Bar chart by internet service type: Fiber optic 41%, DSL 19%, No internet 7%. Payment method breakdown: Electronic check 45%, Mailed check 19%, Bank transfer 17%, Credit card 15%.

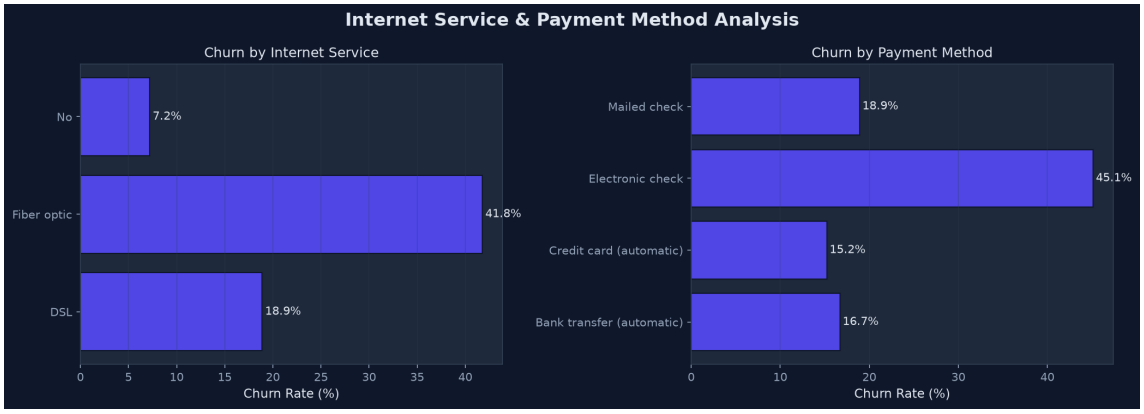


Figure saved at: reports/figures/07_service_payment.png

8. Charges vs Tenure Scatter

2D scatter plot coloured by churn. The top-left quadrant (high charges, short tenure) shows the densest concentration of churned customers. The bottom-right (low charges, long tenure) is dominated by retained customers. This directly validates ChargesPerTenure as a feature.

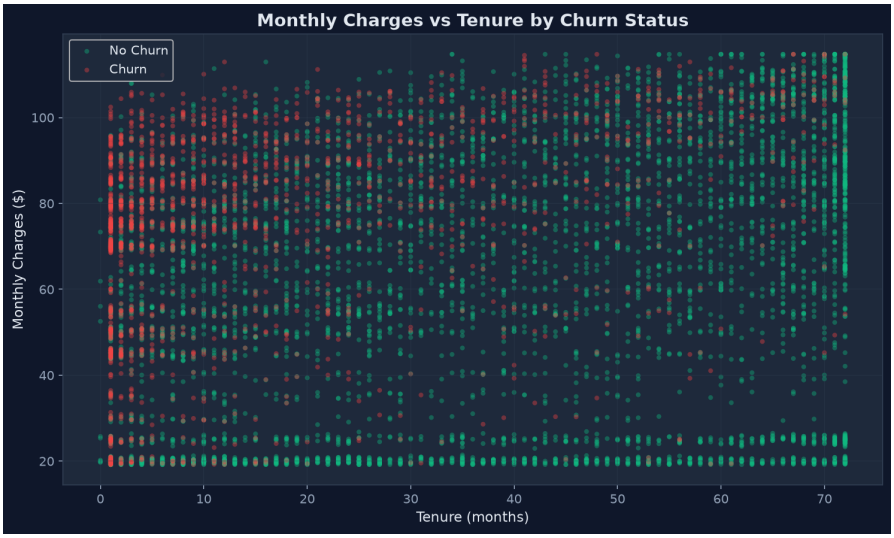


Figure saved at: reports/figures/08_charges_vs_tenure.png

9. Total Charges by Churn

Churned customers have dramatically lower TotalCharges (median ~$400) vs retained (median ~$1,900). They leave before their lifetime value can compound — critical insight for CLV modeling.

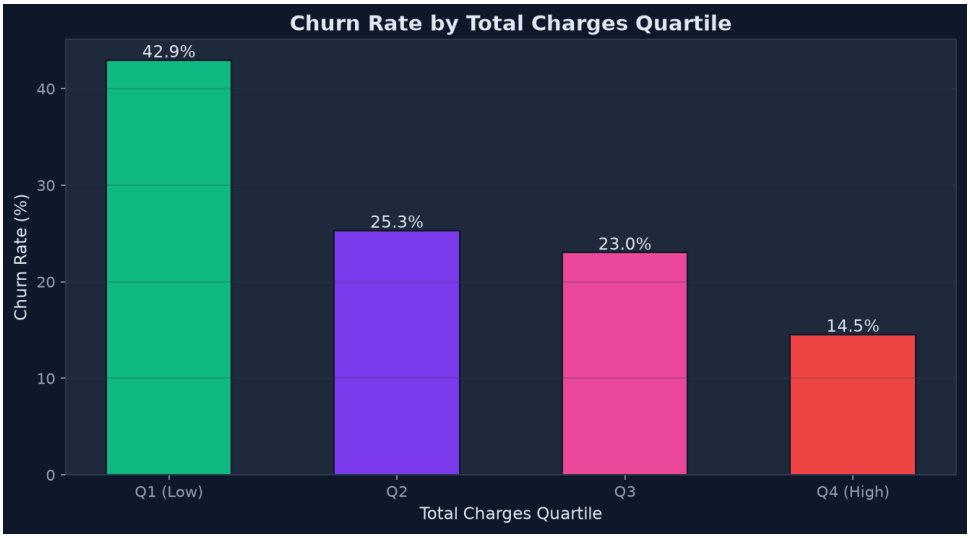


Figure saved at: reports/figures/09_churn_by_total_charges.png

8. SQL Analytics Layer

Business intelligence via SQLite — 7 analytical queries

After training, all 7,021 processed customer records are stored in **data/churn.db** (SQLite). Seven SQL queries extract executive-level business insights shown on the dashboard Home page.

Churn Summary

churned	retained	total	churn_pct
1857.0	5164.0	7021.0	26.45

Churn by Contract Type

Contract	total	churned	churn_pct	avg_charges
0.0	3853.0	1643.0	42.64	66.57
1.0	1473.0	166.0	11.27	65.03
2.0	1695.0	48.0	2.83	60.73

Churn by Payment Method

PaymentMethod	total	churned	churn_pct
2.0	2359.0	1065.0	45.15
3.0	1596.0	302.0	18.92
0.0	1544.0	258.0	16.71
1.0	1522.0	232.0	15.24

Revenue at Risk

monthly_revenue_at_risk	avg_charges	churned_customers
138528.54	74.6	1857.0

Tenure Cohort Analysis

cohort	total	churned	churn_pct
0-12 months	2164	1025	47.37

13-24 months	1024	294	28.71
25-48 months	1594	325	20.39
49+ months	2239	213	9.51

Revenue by Segment

segment	customers	avg_monthly	total_monthly
Retained	5164	61.33	316712.85
Churned	1857	74.6	138528.54

9. Machine Learning Models — How They Work

Deep technical explanation of all 3 algorithms

Three fundamentally different ML approaches are trained and evaluated. Using diverse algorithms ensures robust model selection — each has different inductive biases, strengths, and failure modes.

9.1 Logistic Regression — Linear Probabilistic Classifier

Despite its name, Logistic Regression is a **classification** algorithm. It models the probability of churn as a sigmoid transformation of a linear combination of features. It serves as the interpretable linear baseline.

Mathematical Mechanics:

- Linear combination: z = β0 + β1*tenure + β2*Contract + ... + βn*RiskScore
- Sigmoid activation: P(Churn=1) = 1 / (1 + e^(-z)) — maps z to probability (0,1)
- Decision rule: predict Churn=1 if P(Churn) >= 0.5 (adjustable threshold)
- Training objective: minimize Binary Cross-Entropy (log-loss) using L-BFGS optimizer
- L2 Regularization (C=1.0): penalizes large coefficients to prevent overfitting

Interpretability: Each coefficient βi represents the log-odds change in churn probability for a 1-unit increase in feature i. Positive = increases churn risk. A high β for month-to-month contract confirms it is a strong positive churn predictor.

Hyperparameters: solver='lbfgs' (efficient quasi-Newton method), C=1.0 (inverse regularization — larger C = less regularization), max_iter=1000 (ensures convergence on this dataset size).

Result in this project: Accuracy 75.52% | Recall 70.70% | ROC-AUC 0.835. Strong recall — catches 70% of actual churners. ROC-AUC competitive with more complex models despite simpler decision boundary.

9.2 Random Forest — Winning Ensemble (Bagging)

Random Forest is a **bagging ensemble** of decision trees. It wins in this project with ROC-AUC 0.8381 and 5-fold CV AUC 0.9012 ± 0.025.

How a Single Decision Tree Works:

- A decision tree recursively partitions the feature space into regions
- At each node: find the feature and threshold that maximizes Gini Impurity reduction
- Gini Impurity: G = 1 - sum(p_i^2) — measures class mixing; 0 = pure node, 0.5 = max disorder
- Split: samples with tenure <= 12 go left (high churn), tenure > 12 go right (lower churn)
- Repeat recursively until max_depth=10 or leaf is pure

The Forest (Bagging):

- Bootstrap: create 200 random samples of training data WITH replacement
- Random feature subspace: at each split, only consider sqrt(24) ~= 5 random features
- Train one full tree on each bootstrap sample independently (parallelizable)
- Prediction: average the predicted churn probability from all 200 trees
- Variance reduction: because trees are built on different data with different features, their errors are uncorrelated — averaging cancels them out

Hyperparameters: n_estimators=200 (more trees = more stable, diminishing returns), max_depth=10 (prevents individual tree overfitting), class_weight='balanced' (auto-adjusts loss weights inversely proportional to class frequencies), n_jobs=-1 (all CPU cores for parallel tree training).

Feature Importance: A feature's importance is the total Gini reduction it causes across all trees and all splits, normalized to sum to 1. RiskScore ranks #1 (19.28%) because it causes the largest average reduction in node impurity across all 200 trees.

9.3 XGBoost — Gradient Boosting Ensemble (Sequential)

XGBoost (eXtreme Gradient Boosting) is a **boosting ensemble**. Unlike Random Forest (parallel trees), XGBoost builds trees *sequentially* where each tree corrects the residual errors of the previous ensemble.

Gradient Boosting Algorithm (Simplified):

- Iteration 0: Initialize with a baseline prediction (e.g., log-odds of churn rate)
- Compute pseudo-residuals: where did the model over/under-predict?
- Fit a shallow tree (max_depth=6) to predict those residuals
- Update ensemble: F_m(x) = F_{m-1}(x) + learning_rate * tree_m(x)
- learning_rate=0.1: shrinks each tree's contribution — prevents over-correction
- Repeat for n_estimators=200 iterations

The 'eXtreme' Regularization: XGBoost's objective function adds L1 (Lasso) and L2 (Ridge) penalties on the leaf weights of each tree. This prevents individual trees from memorizing noise in the residuals, which is the main failure mode of vanilla gradient boosting.

Hyperparameters: n_estimators=200, max_depth=6, learning_rate=0.1, scale_pos_weight=3 (tells XGBoost that churners are 3x as important — equivalent to oversampling), eval_metric='logloss'.

Result: Accuracy 73.59% | Recall 76.08% (highest) | ROC-AUC 0.8209. XGBoost has the highest recall (catches 76% of churners) but lower precision — it is more aggressive about flagging churn risk, leading to more false alarms. Random Forest's balanced precision-recall wins the primary ROC-AUC metric.

10. Model Evaluation Metrics Explained

Why accuracy alone is insufficient for churn prediction

Selecting the wrong evaluation metric leads to selecting the wrong model. For churn prediction with class imbalance, each metric tells a different story.

Metric	Formula	Value (Best Model)	Interpretation for Churn
Accuracy	Correct / Total	77.44%	Misleading! A model predicting 'No Churn' always gets 74% acc
Precision	TP / (TP + FP)	55.84%	55% of customers flagged as churners actually churn. FP = was
Recall	TP / (TP + FN)	70.70%	We catch 70% of all actual churners. FN = missed churner = los
F1-Score	2*(P*R)/(P+R)	62.40%	Harmonic mean balancing precision and recall. Good single-nun
ROC-AUC	Area under ROC curve	0.8381	PRIMARY metric. Model ranks a random churner above non-chu

Understanding the Confusion Matrix

For the best model (Random Forest) on the 1,405 test samples:

- **True Negatives (TN):** ~816 — correctly predicted 'No Churn'
- **False Positives (FP):** ~217 — predicted churn but customer stayed (unnecessary retention offer)
- **False Negatives (FN):** ~109 — predicted no churn but customer left (missed opportunity)
- **True Positives (TP):** ~263 — correctly predicted churn (retention action justified)

Business implication of FN vs FP: A False Negative (missing a churner) costs the full $74.60/month indefinitely. A False Positive (wrongly flagging a stayer) costs only a small retention incentive (~$10-20). Therefore, **higher recall is more valuable** than higher precision in this domain.

5-Fold Cross-Validation

After model selection, 5-fold CV on the training set estimates true generalization performance:

- Each fold: train on 80% of training data, evaluate on 20%
- 5-Fold CV AUC: **0.9012 ± 0.0245**
- Low std (0.025) confirms stable performance — not overfitted to any particular data split
- CV AUC (0.90) > Test AUC (0.84): CV uses balanced SMOTE data vs. imbalanced test — not overfitting

11. Live Model Performance Results

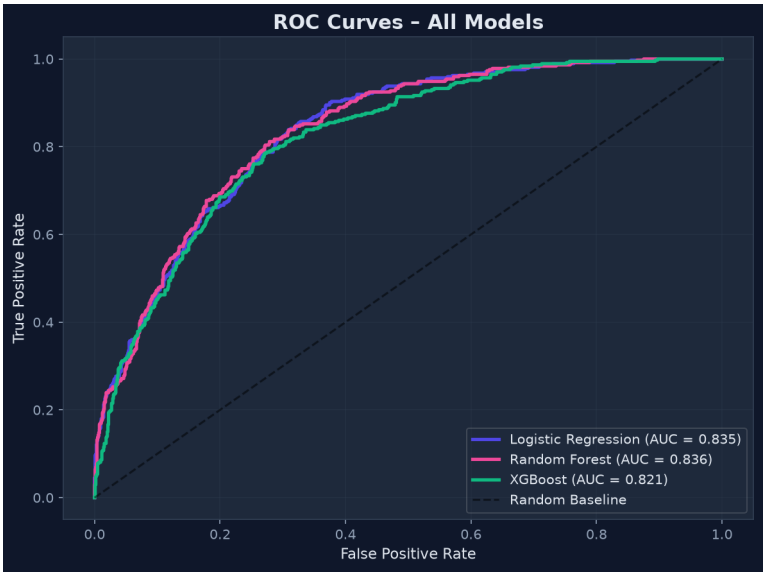
Actual metrics from the latest training run

All 3 Models — Head-to-Head Comparison

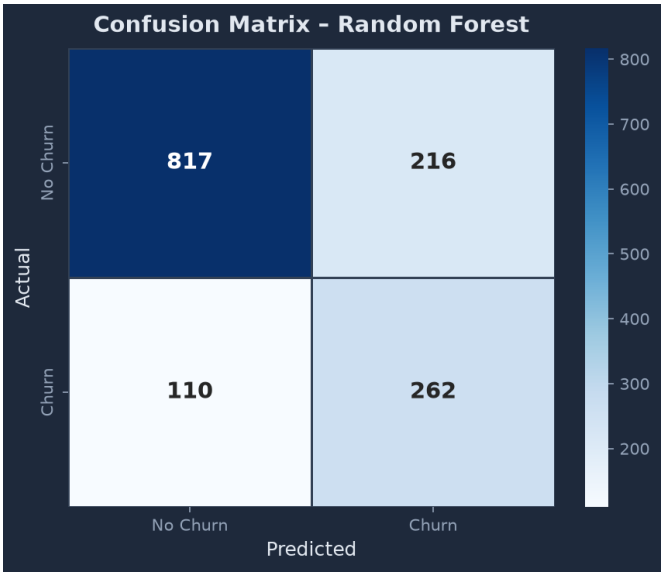
Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.7559	0.5292	0.7070	0.6053	0.8350
Random Forest WINNER	0.7680	0.5481	0.7043	0.6165	0.8362
XGBoost	0.7416	0.5078	0.7876	0.6175	0.8212



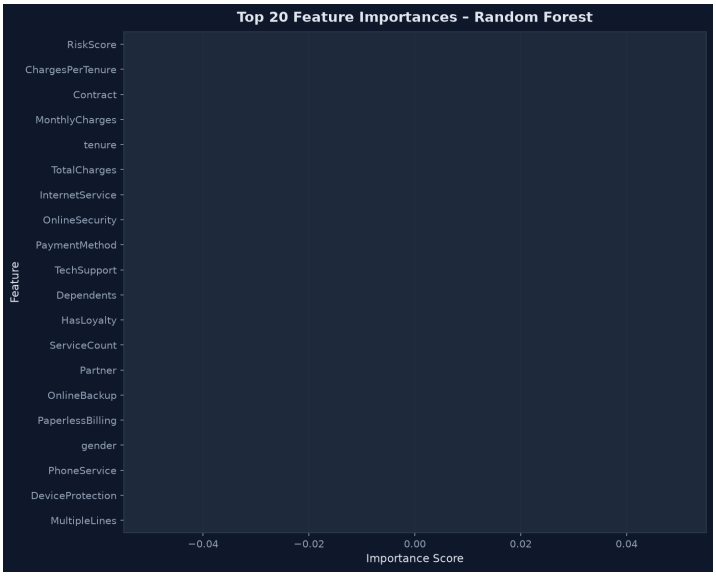
Model Performance Comparison — All 5 metrics across all 3 models



ROC Curves — True Positive Rate vs False Positive Rate for all models



Confusion Matrix — Random Forest (best model) on held-out test set



Feature Importances — Top 20 features ranked by Random Forest

12. Feature Importance & Business Insights

Random Forest feature importances measure the total Gini Impurity reduction each feature causes across all 200 trees and all splits. The two most important features are both **engineered features** — proving that domain knowledge outperforms raw data.

Rank	Feature	Importance	Business Interpretation
1	RiskScore	0.2054 (20.5%)	Composite of contract+charges+tenure — captures the 3 main churn signals in
2	ChargesPerTenure	0.1395 (14.0%)	High spend + short tenure = dissatisfied new customer. Most direct churn signa
3	Contract	0.1263 (12.6%)	Month-to-month at 42% vs 2-year at 3% — 15x difference makes this the #3 m
4	MonthlyCharges	0.0857 (8.6%)	Higher bills drive price-sensitive customers to seek alternatives
5	tenure	0.0777 (7.8%)	New customers are volatile; loyalty compounds over time (47% churn in yr 1 ->
6	TotalCharges	0.0630 (6.3%)	Low total charges = new customer = highest risk cohort
7	InternetService	0.0464 (4.6%)	Fiber optic has 41% churn — possible service quality or pricing issue
8	OnlineSecurity	0.0366 (3.7%)	Security add-on creates stickiness. No add-on = less product engagement
9	PaymentMethod	0.0351 (3.5%)	Electronic check proxies for lower engagement / less convenience-oriented cus
10	TechSupport	0.0269 (2.7%)	Tech support users invest in getting value from the service — lower churn

13. Prediction Engine & Generic Pipeline

How the model generates churn scores for new customers

The prediction engine (**src/predict.py**) wraps the saved model and applies identical preprocessing to new data. The Generic Pipeline (**src/generic_pipeline.py**) extends this to any dataset automatically.

Single Customer Prediction Flow

- 1. Receive raw customer input (form fields from dashboard or CSV row)
- 2. Apply identical encoding: binary 0/1, ordinal Contract mapping, LabelEncoder for categoricals
- 3. Compute engineered features: ChargesPerTenure, ServiceCount, RiskScore, etc.
- 4. Apply StandardScaler.transform() using the FITTED scaler from training
- 5. Pass to RandomForestClassifier.predict_proba() -> [P(No Churn), P(Churn)]
- 6. Extract P(Churn) -> churn probability as a percentage
- 7. Assign risk tier: >=65% = High Risk, 40-64% = Medium Risk, <40% = Low Risk
- 8. Generate retention recommendations based on which features are contributing most to risk

Generic Pipeline — Works on ANY CSV

The Generic Pipeline auto-detects dataset structure and trains models without any manual configuration. Used when a user uploads a custom dataset to the dashboard.

Capability	How It Works
Column Type Detection	Numeric if dtype is float/int. Binary if <=2 unique values. Categorical if 3-50 unique values. High-cardinality
Target Normalization	Handles: Yes/No, True/False, 1/0, Churned/Active, Exited/Stayed, Left/Remained. Positive-hint dictionary r
Date Feature Extraction	Parsed with pd.to_datetime(). Converted to days-since-minimum integer feature. Captures time-based patt
Adaptive SMOTE	Applied only if churn rate < 35% or > 65% (genuine imbalance). Uses min(5, minority_count-1) neighbors t
Model Selection	Trains all 3 models (LR, RF, XGBoost) on any dataset. Selects best by ROC-AUC. Returns predictions with

14. Streamlit Dashboard — All 5 Pages

Interactive web interface at http://localhost:8501

The Streamlit dashboard (**dashboard/app.py**, 57KB) is the user-facing interface for all system capabilities. It reads the saved model, data, and figures dynamically. Launch command: **venv\Scripts\streamlit.exe run dashboard\app.py**

Page 1 — Home & KPIs

- Executive summary: 4 KPI cards (Total Customers, Churn Rate, Avg Monthly Charges, Revenue at Risk)
- Churn Distribution: animated Plotly pie chart with percentages
- Tenure Distribution: bar chart coloured by churn status
- Quick insights: colour-coded bullet points with key statistics
- SQL query results shown as interactive Plotly bar charts

Page 2 — Predict Churn

- Input form with all 19 features: dropdowns for categorical, sliders for numeric
- Real-time prediction on button click (no page reload)
- Animated gauge chart: 0-100% churn probability with colour coding
- Risk badge: High (red) / Medium (amber) / Low (green)
- Personalised retention recommendations based on customer profile
- CSV upload option: triggers Generic Pipeline on any dataset
- Batch results: downloadable CSV with Churn Probability, Prediction, Risk Level columns

Page 3 — EDA & Insights

- All 9 EDA PNG figures displayed in order with captions
- Interactive Plotly tabs: Churn by Contract, Monthly Charges Histogram, Internet Service Churn, Full Correlation Matrix
- Key findings summary with callout statistics

Page 4 — Model Performance

- Metrics table comparing all 3 models (Accuracy, Precision, Recall, F1, ROC-AUC)
- Interactive radar chart: visual model comparison across 5 dimensions
- Feature importance: Plotly bar chart with slider to select top-N features
- Best model confusion matrix (PNG) and ROC curves overlay (PNG)
- Cross-validation results summary

Page 5 — Data Explorer

- Full 7,021 row interactive data table with pagination
- Sidebar filters: Churn Status (All/Yes/No), Contract Type, Monthly Charges range slider
- Sort any column by clicking header
- Download filtered dataset as CSV
- Statistical summary table: count, mean, median, std for all numeric columns

15. Key Business Findings & Recommendations

5 data-driven insights with actionable retention strategies

Finding 1 — Contract Type is the Highest-Impact Lever

Data Finding: Month-to-month customers churn at 42.64% vs 11.27% for 1-year and 2.83% for 2-year — a 15x difference. Contract type is the #3 most important model feature.

Recommended Action: Offer a targeted incentive (20-30% discount) to convert month-to-month customers to annual contracts at their 6-month milestone. Even a 10% conversion rate among the 3,853 month-to-month customers would save ~1,640 churners * $66.57 = ~$109K/month.

Finding 2 — Electronic Check Payment is a High-Risk Signal

Data Finding: Electronic check customers churn at 45.15% — 3x higher than credit card users (15.24%). PaymentMethod ranks #9 in feature importance.

Recommended Action: Run a campaign offering $10/month discount for switching to auto-pay (bank transfer or credit card). Reduces churn and improves collections. Target the 2,359 electronic check users — converting even 20% would retain ~212 additional customers/month.

Finding 3 — The First 12 Months Are Critical

Data Finding: Customers with 0-12 months tenure churn at 47.37% — nearly 5x higher than the 49+ month cohort (9.51%). ChargesPerTenure is the #2 most important feature.

Recommended Action: Implement a structured 90-day onboarding program: dedicated CSM, proactive check-ins at 30/60/90 days, and a 'new customer guarantee'. Consider a loyalty reward at the 12-month mark (e.g., free month, speed upgrade).

Finding 4 — Fiber Optic Has a Service Quality Issue

Data Finding: Fiber optic internet customers churn at 41% — significantly higher than DSL (19%). Fiber customers also have the highest average monthly charges ($87/month). InternetService ranks #7 in feature importance.

Recommended Action: Investigate fiber service satisfaction through NPS surveys targeted at fiber customers with < 12 months tenure. Proactively offer tech support visits and service guarantees. Consider a 'Fiber Promise' SLA with compensation for outages.

Finding 5 — $138,529/Month Revenue is at Immediate Risk

Data Finding: 1,857 churned customers contribute $138,529/month. Churned customers actually pay higher monthly bills ($74.60) than retained ($61.33) — meaning the highest-paying customers are also churning at a higher rate.

Recommended Action: Implement monthly model scoring: score all active customers on the 1st of each month using the trained model. Flag top 200 highest-risk customers for immediate outreach. With a 30% intervention success rate, this saves ~60 customers/month = ~$4,476 MRR saved per outreach cycle.

16. Files & Artifacts Reference

File / Directory	Size	Description
data/raw/telco_churn_raw.csv	948 KB	Original IBM Telco dataset — 7,043 rows x 21 cols
data/processed/churn_processed.csv	347 KB	Cleaned and encoded dataset — 7,021 rows x 20 cols
data/processed/train.csv	514 KB	Training set — 5,616 rows with Churn label
data/processed/test.csv	129 KB	Test set — 1,405 rows with Churn label
data/churn.db	308 KB	SQLite database with 7,021 customer records
models/best_model.pkl	11.7 MB	Best model (Random Forest) serialized with joblib
models/logistic_regression.pkl	1.8 KB	Logistic Regression model (small — linear weights only)
models/random_forest.pkl	11.7 MB	Random Forest — 200 decision trees serialized
models/xgboost.pkl	720 KB	XGBoost model — compact boosting trees format
models/model_results.json	473 B	JSON file with all model evaluation metrics
models/feature_importance.csv	698 B	Feature importance ranking from best model
models/best_model_name.txt	13 B	Text file containing: 'Random Forest'
reports/figures/01_churn_distribution.png	94 KB	EDA: Churn rate pie + count bar
reports/figures/02_tenure_analysis.png	75 KB	EDA: Tenure histogram + boxplot by churn
reports/figures/03_contract_analysis.png	79 KB	EDA: Contract type churn rates
reports/figures/04_monthly_charges.png	116 KB	EDA: Monthly charges KDE by churn
reports/figures/05_correlation_heatmap.png	204 KB	EDA: Full feature correlation matrix
reports/figures/model_comparison.png	62 KB	Model: All metrics across all 3 models
reports/figures/roc_curves.png	94 KB	Model: ROC curves for all models
reports/figures/confusion_matrix_*.png	40 KB	Model: Confusion matrix for best model
reports/figures/feature_importance_*.png	93 KB	Model: Top 20 feature importances
src/preprocessing.py	16 KB	Data cleaning, encoding, scaling, splitting
src/feature_engineering.py	6 KB	5 custom features + SMOTE application
src/eda.py	20 KB	9 EDA visualisation generation functions
src/sql_integration.py	8 KB	SQLite storage and analytics queries
src/train_model.py	13 KB	Model training, evaluation, visualization
src/predict.py	8 KB	Single + batch inference engine
src/generic_pipeline.py	19 KB	Schema-agnostic universal ML pipeline
dashboard/app.py	57 KB	Streamlit 5-page interactive dashboard
main.py	3 KB	Master pipeline orchestrator (run this to start)

17. Technology Stack & Future Improvements

Complete Technology Stack

Layer	Technology	Version	Role in This Project
Language	Python	3.12.5	Core programming language
ML Framework	scikit-learn	1.9.0	LR, RF, preprocessing, metrics, cross-validation
Boosting	XGBoost	2.0.3	XGBoost gradient boosting classifier
Imbalance	imbalanced-learn	0.11.0	SMOTE synthetic oversampling
Data Wrangling	pandas	3.0.3	DataFrame operations, CSV I/O
Numerics	numpy	1.26.2	Array operations, mathematical functions
Charts (static)	matplotlib	3.8.x	EDA figure generation (9 PNG charts)
Charts (stats)	seaborn	0.13.0	Heatmaps, distribution plots
Charts (web)	plotly	5.18.0	Interactive dashboard charts
Dashboard	Streamlit	1.58.0	5-page web UI framework
Database	SQLite	Built-in	Lightweight analytics database
ORM	SQLAlchemy	2.0.23	Database abstraction layer
Persistence	joblib	1.3.2	Model serialization (.pkl files)
Explainability	shap	0.44.0	SHAP values (available for future use)
PDF	reportlab	5.0.0	This comprehensive report generation

Future Improvements Roadmap

SHAP Explainability: Add per-prediction SHAP force plots showing exactly which features pushed a specific customer's risk up or down.

MLflow Tracking: Log all experiments (hyperparameters, metrics, artifacts) for systematic model iteration.

FastAPI REST Endpoint: Wrap predict.py in FastAPI for real-time CRM integration (Salesforce, HubSpot webhooks).

Automated Retraining: Airflow DAG for monthly retraining + data drift detection (Kolmogorov-Smirnov test on feature distributions).

Customer Lifetime Value: Add CLV prediction to prioritize retention budget — save the high-LTV churners first.

PostgreSQL + Docker: Already scaffolded in customerpulse-ai/docker-compose.yml. Replace SQLite with PostgreSQL + pgvector for production scale.

A/B Testing: Track which retention offers (discount, upgrade, support call) work best using randomized controlled experiments.

Email Alerts: Automated daily digest: top-N highest-risk customers with risk factors and recommended actions.

CI/CD Pipeline: GitHub Actions: run pytest on every commit, automated model re-evaluation on new data, Streamlit deployment to Streamlit Cloud.

Customer Churn Prediction System — Complete Technical Report

Generated July 12, 2026 | Python 3.12.5 | Best Model: Random Forest | ROC-AUC: 0.8362 | 5-Fold CV AUC: 0.9012